

Project Charter — SafeExec

A Hardened, Threat-Modeled Python Execution Sandbox for LLM Agent Tool-Use

Field	Value
Working title	SafeExec
Student	Zixuan Liang (zliang1@my.harrisburgu.edu)
Program	M.S. Computer Information Sciences, Harrisburg University
Course	CISC 699-50-A-2026/Summer — Applied Project in Computer Information Sciences
Supervisor	Dr. Majid Shaalan
Term	2026-05-09 → 2026-08-14 (14 weeks)
Charter draft date	2026-05-13
Charter approval target	W2 (2026-05-22)

1. Rationale

Agentic systems built on large language models increasingly rely on a tool-use loop in which the model proposes code, an executor runs it, and the result is fed back into the model's next step. In production, the executor is the part most often built quickly and revisited never. The result is that real systems today execute model-generated code with isolation that ranges from "an unsandboxed subprocess inside the agent's own process tree" to "a Docker container with default settings and a permissive network policy." Neither is appropriate when the code being executed was produced by an adversarial-input-susceptible language model under conditions of possible prompt injection.

This is a meaningful applied problem — directly relevant to agent platforms now in production at multiple organizations — and it is one in which the gap between what exists and what would be defensible is small enough that a single graduate student can credibly close it for a focused workload (Python, single-tenant, no network, no GPU) over fourteen weeks. The work is *not* a tutorial replication; existing open-source projects in the space (E2B, Daytona, Modal sandboxes) cover adjacent slices but do not publish documented threat models tied to reproducible adversarial benchmarks. The intellectual contribution is the **adversarial test suite plus methodology** — a measurement framework that lets the isolation strength of a sandbox be evaluated empirically rather than asserted.

This problem is appropriate for CISC 699 because it produces (a) a working computational artifact with non-trivial systems content, (b) an empirical evaluation with statistical structure, (c) a defensible ethics/security/broader-impact story, and (d) reproducibility artifacts (Dockerfiles, compose files, pinned versions, deterministic test runners) that can be exercised by any reader on commodity hardware.

2. Intended artifact

The project will deliver a single integrated artifact comprising:

1. **The sandbox execution service** ([src/](#)): a small HTTP service exposing a `POST /execute` endpoint that accepts Python source, stdin, and a timeout, and returns stdout, stderr, exit code, wall-clock duration, and peak memory. Internally, the service supports two pluggable isolation back-ends through a common executor interface.
2. **Hardened Docker back-end** (W5–W6 milestone): rootless, read-only root filesystem, no network namespace, dropped Linux capabilities, custom seccomp filter, AppArmor or SELinux profile, cgroup-enforced limits on CPU, memory, PIDs, file descriptors, and wall-clock.
3. **gVisor back-end** (W7–W8 milestone): same hardened configuration, plus gVisor's user-space kernel for syscall interposition.
4. **Threat model document** ([docs/design/threat-model.md](#)): named adversary capabilities and named defenses, with explicit out-of-scope items.
5. **Functional test suite** ([tests/functional/](#)): ≥100 Python programs whose output must match a reference Python 3.11 interpreter.
6. **Adversarial test suite** ([tests/adversarial/](#)): ≥40 curated programs across ≥6 categories (resource exhaustion, fd exhaustion, fork bombs, attempted syscall abuse, attempted persistence, host enumeration, attempted exfiltration). Each program ships with an expected contained-outcome label.
7. **Performance benchmark harness** ([benchmarks/](#)): cold-start, warm-start, and steady-state latency; memory overhead; throughput. Reported with confidence intervals.
8. **Minimal reference agent** ([scripts/demo_agent.py](#)): a small Claude-API-driven agent that uses the sandbox as a tool, included strictly as integration demo.
9. **Reproducibility materials**: pinned Dockerfiles, [docker-compose.yaml](#), Makefile targets, deterministic test runner, environment file, README setup section.
10. **Technical report** ([docs/reports/final-report.md](#)): 15–25 pages excluding appendices, structured per CLAUDE.md.

3. Likely users

- **Primary.** LLM-agent developers in industry or academia who need a defensible Python execution layer and currently roll their own.
- **Secondary.** Security engineers evaluating agent platforms; researchers extending the adversarial benchmark; instructors teaching agent-systems courses.
- **Implicit.** The student's future hiring audience — engineers and managers on agent-tooling and product teams at AI labs, who will examine the artifact and report as part of portfolio review.
- **Course audience.** Supervisor (Dr. Shaalan), CISC 699 graders, and final-presentation attendees.

4. Goals

Primary.

1. Deliver a working sandbox service with the two named back-ends, satisfying the functional suite at ≥99%.
2. Author and publish an adversarial test suite of ≥40 programs across ≥6 categories with documented expected outcomes.
3. Produce an empirical comparison of isolation strength and performance overhead across the two back-ends, with statistical confidence intervals.

4. Produce a 15–25 page graduate-level technical report meeting the rubric across all five dimensions.

Secondary. 5. Document a threat model that names what the sandbox does and does not defend against. 6. Ship a minimal reference agent demonstrating end-to-end tool-use integration. 7. Maintain reproducibility such that a reader can execute `docker compose up && make test && make bench` on Ubuntu 22.04 or 24.04 and reproduce reported numbers within 10%.

Stretch (only if ahead at W10). 8. Add a Firecracker microVM back-end as a third comparison point. 9. Release the adversarial benchmark as a standalone open-source artifact.

5. Scope boundaries

In scope

- Python 3.11 user code execution.
- Synchronous request/response API (`POST /execute`).
- Ephemeral per-request filesystem.
- Resource limits: CPU, memory, wall-clock, PIDs, file descriptors.
- Two isolation back-ends: hardened Docker, gVisor.
- Linux host (Ubuntu 22.04 or 24.04, x86-64).
- Functional, adversarial, and performance test suites.
- Documented threat model.
- Minimal reference agent for demo only.

Out of scope (explicitly)

- Multi-language support beyond Python (no JS, R, bash, etc.).
- Network access from inside the sandbox.
- GPU access.
- Persistent state or sessions across requests.
- Multi-tenant security (mutually distrusting users on one instance).
- Production-grade deployment hardening: TLS, authentication, rate limiting beyond demo-grade.
- Windows or macOS hosts.
- Distributed scheduling, Kubernetes operators, or production orchestration.
- Firecracker back-end (stretch only).
- Side-channel timing attacks, Spectre/Meltdown-class attacks, hardware attacks.
- Defenses against nation-state-grade adversaries.

Scope-fence policy

The scope above is fixed for the duration of W3–W8. Any proposed change must be reviewed against the milestone map and approved by the supervisor in writing. The W7 midpoint review is the explicit checkpoint for scope renegotiation; outside of that checkpoint, the default is no change.

6. Assumptions

- A Linux host (local VM, cloud VM, or HU lab) with kernel ≥ 5.10 is available throughout the term.
- Docker 24+ and gVisor (`runs`) can be installed on the chosen host. (Verified prior to W2.)

- The student has Anthropic API access (Claude) for the reference-agent demo. Budget envelope is ~\$50 for the demo.
- Cloud compute budget envelope for isolation experiments: ~\$200–\$400 across the term.
- No regulated or proprietary data is processed through the sandbox during evaluation.
- The student is the sole author of code committed to the repository, with AI assistance disclosed in [docs/ai-use-log.md](#).
- Frontier-model APIs remain available and behave consistently enough across the term to support a small reference demo.

7. Stakeholders

Stakeholder	Relationship	Expectations
Student (Zixuan Liang)	Builder, author, reporter	Owns all decisions, produces all artifacts, manages timeline
Supervisor (Dr. Majid Shaalan)	Approves charter; reviews milestones; final grader	Receives charter, midpoint demo, final report and artifact, final presentation
CISC 699 program	Sets standards and rubric	Adherence to milestone calendar and submission conventions
Future portfolio reviewers	Implicit audience	A defensible artifact and report suitable to discuss in a 30-minute interview
Open-source community (potential)	Downstream consumer if released	Documentation, license clarity, reproducibility

8. Risks and mitigations

#	Risk	Likelihood	Impact	Mitigation	Owner
R1	Scope creep into multi-language or persistence features	Medium	High	Hard scope-fence in this charter; W7 explicit checkpoint	Student
R2	gVisor performance penalty too variable for meaningful comparison	Medium	Medium	Define benchmark protocol in design phase (W3–W4); pre-register sample sizes and percentile-based reporting	Student
R3	Adversarial suite too small or unrepresentative	Medium	High	Define category taxonomy at W3; commit to ≥ 40 programs across ≥ 6 categories; midpoint peer review	Student + Supervisor
R4	Time loss to container-runtime debugging	Medium	Medium	W5 hard deadline for "hello world execution"; if missed, pivot to Docker-	Student

#	Risk	Likelihood	Impact	Mitigation	Owner
				only and drop gVisor to stretch	
R5	Linux host/kernel mismatch between dev and grader environments	Low	Medium	Pin a single Ubuntu LTS image; document environment in README	Student
R6	API cost overrun for reference-agent demo	Low	Low	Cap demo runs; cache transcripts; switch to Haiku if needed	Student
R7	AI-use disclosure gaps if log isn't kept current	Low	High	Update docs/ai-use-log.md at end of every AI-assisted session; weekly check during W11 report-drafting	Student
R8	Supervisor expectations diverge from charter	Low	High	W2 explicit charter approval meeting; W7 mid-course review	Student + Supervisor
R9	Reproducibility fails for the grader (e.g., they can't run Docker)	Low	Medium	Provide a recorded demo + pre-rendered result tables as fallback artifacts	Student
R10	Negative or null results in performance comparison	Medium	Low	Plan the report's W9 section to interpret null results as findings rather than failures	Student

9. Success criteria

The project will be considered successful if all of the following hold at submission:

Functional. The sandbox executes the ≥ 100 -program functional suite with output matching the reference Python 3.11 interpreter on $\geq 99\%$ of programs under both back-ends.

Isolation. The sandbox correctly contains (blocks, terminates within limit, or restricts per spec) $\geq 95\%$ of adversarial test programs under both back-ends. Any non-contained outcomes are explicitly documented as known limitations.

Performance. Cold-start latency, warm-start latency, and steady-state per-request latency are reported for both back-ends with 95% confidence intervals over ≥ 30 samples per condition. Performance overhead between back-ends is reported with explicit interpretation.

Reproducibility. A graduate-level reader on a stock Ubuntu 22.04 or 24.04 host with Docker installed can clone the repo, run `make setup` & `make test` & `make bench`, and obtain reported numbers within 10% on equivalent hardware.

Communication. The final technical report meets rubric Level 1 (Advanced/Exceptional, 90–100%) across all five RU-01 dimensions. The final presentation runs end-to-end in under 20 minutes and the live demo completes in under 5 minutes.

Process. The engineering log has weekly entries from W1 onward. The annotated bibliography reaches ≥ 15 sources by W11. The AI-use disclosure log is complete and consolidated into the report's appendix by W13.

10. Milestone map

Week	Date range	Deliverable
W1	2026-05-09 → 2026-05-15	Launch packet: charter, problem statement, candidates, feasibility memo, bibliography starter (≥ 5), supervisor briefing
W2	2026-05-16 → 2026-05-22	Charter approval; project plan submitted; success criteria finalized
W3	2026-05-23 → 2026-05-29	Related-work synthesis; requirements & use cases; data/source notes
W4	2026-05-30 → 2026-06-05	Design review: architecture, threat model, evaluation plan approved
W5	2026-06-06 → 2026-06-12	Implementation sprint I — Docker back-end, basic API, first functional test passes
W6	2026-06-13 → 2026-06-19	Implementation sprint II — hardened Docker (seccomp, AppArmor, cgroups), functional suite ≥ 50 programs
W7	2026-06-26	Midpoint demo / progress review. Functional suite ≥ 100 , adversarial taxonomy locked, ≥ 20 adversarial programs
W8	2026-06-27 → 2026-07-03	Implementation sprint III — gVisor back-end, full adversarial suite ≥ 40 programs
W9	2026-07-04 → 2026-07-10	Results, limitations, error analysis, broader-impact draft
W10	2026-07-11 → 2026-07-17	Artifact hardening: README, setup, reproducibility check on clean VM
W11	2026-07-18 → 2026-07-24	Full draft of technical report due for supervisor feedback
W12	2026-07-25 → 2026-07-31	Revision cycle; presentation deck; demo script
W13	2026-08-01 → 2026-08-07	Presentation rehearsal; final package check; AI-use appendix consolidated
W14	2026-08-08 → 2026-08-14	Final report, artifact, and presentation submission; reflection

11. Open questions for supervisor (W2 charter-approval meeting)

These are captured in detail in [supervisor-briefing.md](#); summarized here for charter visibility.

- Is the scope (Python-only, two back-ends, ≥ 40 -program adversarial suite) sized correctly for CISC 699 expectations, or should it be widened/narrowed?
- Is the adversarial-suite-as-contribution framing acceptable as the project's "intellectual contribution" dimension, or does the supervisor prefer a different angle?
- Are there preferred citation conventions, deck templates, or report templates for the program?
- Is a private GitHub repository acceptable for the artifact submission, or does the program prefer a different version-control arrangement?

12. Approval

Role	Name	Signature / Date
Student	Zixuan Liang	_____
Supervisor	Prof. Majid Shaalan	_____